Algorithmic Analysis HW 1

Luke L

October 2026

1. Arrange the following list of functions in ascending order of growth rate. Note that two functions may have the same asymptotic growth rate! Prove the relationship of every pair of adjacent functions in your ordered list, even those functions that have the same growth rate.

   $f_a(n) = (n+1) + n + (n-1) + (n-2) \cdots + 2 + 1$
   $f_b(n) = 2^{2n}$
   $f_c(n) = \log n^2$
   $f_d(n) = n!$
   $f_e(n) = 4^n + n$
   $f_f(n) = n \log n$
   $f_g(n) = \sqrt{n}$

   We begin by simplifying three of the functions. First, f_a is the sum of integers from 1 to m, $m = n + 1$, so we use the summation formula.

   $$\begin{aligned} f_a(n) &= (n+1) + n + (n-1) + (n-2) + \cdots + 2 + 1 \\ &= \frac{m(m+1)}{2} \\ &= \frac{(n+1)(n+2)}{2} = \Theta(n^2) \end{aligned} \tag{1}$$

   Then, two minor simplifications to f_b and f_c.

   $$f_b(n) = 2^{2n} = (2^2)^n = 4^n \tag{2}$$
   $$f_c(n) = \log n^2 = 2 \log n = \Theta(\log n) \tag{3}$$

   We now prove the ordering:

   $$f_c < f_g < f_f < f_a < (f_b, f_e) < f_d$$

   Proof. For functions that are positive for large values of n, if

   $$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0,$$

then $f(n) = o(g(n))$, so f has strictly smaller growth. If the limit is a positive finite constant, then $f(n) = \Theta(g(n))$.

We check all six adjacent pairs. For the limits involving logarithms, use $\log n = \ln n / \ln b$, where $b > 1$. L'Hopital's Rule requires you to take derivatives. It's much easier to take the derivative of the natural log than a generic log.

(a) $f_c(n) = o(f_g(n))$. By L'Hopital's rule,

$$\lim_{n\to\infty} \frac{2\log n}{\sqrt{n}} = \frac{2}{\ln b} \lim_{n\to\infty} \frac{1/n}{1/(2\sqrt{n})} = \frac{4}{\ln b} \lim_{n\to\infty} \frac{1}{\sqrt{n}} = 0.$$

(b) $f_g(n) = o(f_f(n))$.

$$\lim_{n\to\infty} \frac{\sqrt{n}}{n\log n} = \lim_{n\to\infty} \frac{1}{\sqrt{n}\log n} = 0.$$

(c) $f_f(n) = o(f_a(n))$. First, L'Hopital's rule gives

$$\lim_{n\to\infty} \frac{\log n}{n} = \frac{1}{\ln b} \lim_{n\to\infty} \frac{1}{n} = 0.$$

Therefore,

$$\lim_{n\to\infty} \frac{n\log n}{(n+1)(n+2)/2} = \lim_{n\to\infty} \frac{2(\log n)/n}{(1+1/n)(1+2/n)} = 0.$$

(d) $f_a(n) = o(f_b(n))$. Applying L'Hopital's rule twice,

$$\lim_{n\to\infty} \frac{(n+1)(n+2)}{2\cdot 4^n} = \lim_{n\to\infty} \frac{2n+3}{2(\ln 4)4^n} = \lim_{n\to\infty} \frac{2}{2(\ln 4)^2 4^n} = 0.$$

(e) $f_b(n) = \Theta(f_e(n))$. Since

$$\lim_{n\to\infty} \frac{n}{4^n} = \lim_{n\to\infty} \frac{1}{(\ln 4)4^n} = 0,$$

we obtain

$$\lim_{n\to\infty} \frac{4^n}{4^n+n} = \lim_{n\to\infty} \frac{1}{1+n/4^n} = 1.$$

Thus, these functions have the same asymptotic growth rate.

(f) $f_e(n) = o(f_d(n))$. To prove $f_e(n) = o(f_d(n))$, we must show that

$$\lim_{n\to\infty} \frac{4^n+n}{n!} = 0.$$

We separate the ratio into two terms:

$$\frac{4^n+n}{n!} = \frac{4^n}{n!} + \frac{n}{n!}.$$

First, consider $4^n/n!$. At each step, the numerator gains a factor of 4, while the denominator gains an increasing factor. For $n \geq 8$, (it is possible to lower this bound) we can write

$$\frac{4^n}{n!} = \frac{4^7}{7!} \underbrace{\frac{4}{8} \cdot \frac{4}{9} \cdot \frac{4}{10} \cdots \frac{4}{n}}_{n-7 \text{ factors}}.$$

Each of these $n-7$ factors is at most $1/2$. Therefore,

$$0 \leq \frac{4^n}{n!} \leq \frac{4^7}{7!} \left(\frac{1}{2}\right)^{n-7}.$$

The factor $4^7/7!$ is constant, and $(1/2)^{n-7} \to 0$. Thus, by the squeeze theorem,

$$\lim_{n\to\infty} \frac{4^n}{n!} = 0.$$

For the second term, cancel the factor n:

$$\frac{n}{n!} = \frac{1}{(n-1)!} \longrightarrow 0.$$

Combining these limits gives

$$\lim_{n\to\infty} \frac{4^n + n}{n!} = \lim_{n\to\infty} \frac{4^n}{n!} + \lim_{n\to\infty} \frac{1}{(n-1)!} = 0 + 0 = 0.$$

Therefore, $f_e(n) = o(f_d(n))$.

□

2. Given functions f and g where $f(n)$ is $O(g(n))$, determine if the following statements are true or false. Support your answer via definition of big-O or counter-example.

   a) $2^{f(n)}$ is $O(2^{g(n)})$ This is false.

   Proof. We assume this is true. Let $f(n) = 2n$ and $g(n) = n$. Then $f(n) \in O(n) = O(g(n))$. But $2^{f(n)} = 2^{2n} = 4^n$, and $2^{g(n)} = 2^n$. For 4^n to be $O(2^n)$, from the definition of Big Oh, we need a constant c such that $4^n \leq c2^n$. Dividing by 2^n gives $2^n \leq c$ for all sufficiently large n. This is impossible because $2^n \to \infty$. □

   b) $\sqrt{n} * f(n)$ is $O(f(n) * g(n))$ This is also false.

   Proof. Assume this is true. Let $f(n) = 1$ and $g(n) = 1$. Then $f(n) \in O(g(n)) = O(1)$. So $\sqrt{n} * 1 \in O(f(n) * g(n)) = O(1)$. But this would require $\sqrt{n} \leq c$ for all sufficiently large n. This is impossible because $\sqrt{n} \to \infty$. □

3. Put the given recurrence relations into closed form: State any assumptions made. Do NOT use the Master Theorem. Answer the question in part c), providing supportive detail.

   (a) We are given:

$$T(n) = 2T(n-1) + n$$
$$T(1) = 2$$

Starting with the recurrence, and assuming n is a positive integer,

$$T(n) = 2T(n-1) + n,$$

expand it repeatedly:

$$\begin{aligned} T(n) &= 2\big(2T(n-2) + (n-1)\big) + n \\ &= 2^2 T(n-2) + 2(n-1) + n \\ &= 2^2\big(2T(n-3) + (n-2)\big) + 2(n-1) + n \\ &= 2^3 T(n-3) + 2^2(n-2) + 2(n-1) + n. \end{aligned}$$

After j expansions, the pattern is

$$T(n) = 2^j T(n-j) + \sum_{i=0}^{j-1} 2^i (n-i).$$

Set $j = n-1$, so that $T(n-j) = T(1)$:

$$\begin{aligned} T(n) &= 2^{n-1} T(1) + \sum_{i=0}^{n-2} 2^i (n-i) \\ &= 2^n + \sum_{i=0}^{n-2} 2^i (n-i), \end{aligned}$$

since $T(1) = 2$.
For $n \geq 2$, evaluate the sum from this simplification:

$$\sum_{i=0}^{n-2} 2^i (n-i) = n \sum_{i=0}^{n-2} 2^i - \sum_{i=0}^{n-2} i 2^i.$$

Use the formulas

$$\sum_{i=0}^{m} 2^i = 2^{m+1} - 1 \qquad \text{and} \qquad \sum_{i=0}^{m} i 2^i = (m-1)2^{m+1} + 2.$$

Setting $m = n-2$, we obtain

$$\sum_{i=0}^{n-2} 2^i = 2^{n-1} - 1 \qquad \text{and} \qquad \sum_{i=0}^{n-2} i 2^i = (n-3)2^{n-1} + 2.$$

Therefore,

$$\sum_{i=0}^{n-2} 2^i(n-i) = n(2^{n-1}-1) - \left((n-3)2^{n-1}+2\right)$$
$$= 3 \cdot 2^{n-1} - n - 2.$$

Substitute this into the expression for $T(n)$:

$$T(n) = 2^n + 3 \cdot 2^{n-1} - n - 2$$
$$= 5 \cdot 2^{n-1} - n - 2.$$

The formula also gives $T(1) = 2$, as required. Since the exponential term dominates the linear term, the growth rate is

$$\boxed{T(n) = \Theta(2^n).}$$

(b) We are given:

$$T(n) = 4T(n/2) + 1$$
$$T(1) = 4$$

Assume $n = 2^k$ for some nonnegative integer k, so each recursive call has an integer input. Starting with

$$T(n) = 4T\left(\frac{n}{2}\right) + 1,$$

expand the recurrence:

$$T(n) = 4\left(4T\left(\frac{n}{4}\right) + 1\right) + 1$$
$$= 4^2 T\left(\frac{n}{4}\right) + 4 + 1$$
$$= 4^2\left(4T\left(\frac{n}{8}\right) + 1\right) + 4 + 1$$
$$= 4^3 T\left(\frac{n}{8}\right) + 4^2 + 4 + 1.$$

After j expansions, the pattern is

$$T(n) = 4^j T\left(\frac{n}{2^j}\right) + \sum_{i=0}^{j-1} 4^i.$$

Set $j = k = \log_2 n$, so that $n/2^j = 1$. Then

$$\begin{aligned} T(n) &= 4^k T(1) + \sum_{i=0}^{k-1} 4^i \\ &= 4^k \cdot 4 + \frac{4^k - 1}{4 - 1} \\ &= 4n^2 + \frac{n^2 - 1}{3} \\ &= \frac{13n^2 - 1}{3}, \end{aligned}$$

because $4^k = (2^k)^2 = n^2$. We have omitted the result of computing a sum $S = \sum_{i=0}^{k-1} 4^i$ by subtracting it from $4S$. The formula also gives $T(1) = 4$, as required.

Since the quadratic term dominates the constant term, the growth rate is

$$\boxed{T(n) = \Theta(n^2).}$$

(c) Explain what each result says about the cost of a corresponding Divide and Conquer algorithm. In part (a), each call makes two calls of size $n - 1$. The input decreases slowly while the number of calls grows exponentially, giving $\Theta(2^n)$ time.

In part (b), each call makes four calls of size $n/2$. The expansion shows that the total cost is $\Theta(n^2)$. Whether this improves on another algorithm depends on the problem and the alternative algorithm. This result shows that the complexity of a) is exponential, and risks being intractable. Maybe dividing up the problems or input space is not the most optimal decision in this case. For b), without more information we can't conclude if this algorithm is a good use of divide and conquer. If we improved from cubic to quadratic growth, that is great, but if we are solving a problem with $\Omega(n^2)$ complexity, and brute force solutions give the same growth rate, we probably didn't improve it very much with this new algorithm.

4. Consider the following solutions to two classic Divide Conquer problems
`https://www.geeksforgeeks.org/tiling-problem-using-divide-and-conquer-algorithm/`
`https://www.gregegan.net/SCIENCE/FindTheFakeCoin/FindTheFakeCoin.html`

(a) **Explain the solutions.**

Tiling problem. Let the board have dimensions $n \times n$, where $n = 2^k$, with one missing cell.

i. Divide the board into four equal quadrants.

ii. Place one L-shaped tile at the center. It covers one central cell in each quadrant without the missing cell.

iii. Each quadrant now has exactly one cell that must not be covered by its recursive tiling.

iv. Recursively tile all four quadrants. A 2×2 board is solved with one tile. This is the base case.

This construction avoids searching through all possible tile placements. The four smaller quadrants that have been tiled, together with the central tile, form the complete tiling.

Fake-coin problem. First assume that the fake coin is known to be lighter. We call possible fake coins "candidates." Divide the candidates into three nearly equal groups, with the first two equal in size. Weigh those two groups:

- If they balance, keep the third group. The two groups are balanced and contain no fakes, so we isolate the potential fake group.
- Otherwise, keep the lighter group.

Repeat until one candidate remains. For a known heavier fake coin, keep the heavier group instead. Each weighing eliminates approximately two-thirds of the candidates.

If the weight difference is unknown, Greg Egan's solution tracks three groups: coins that may be lighter fakes, coins that may be heavier fakes, and coins known to be genuine. He chooses each weighing to divide the remaining possibilities among the three possible outcomes: the left pan is lighter, the coins have balance, or the left pan is heavier. After the weighing, he follows the branch that matches the observed outcome. The choices are more complex than a simple three-way split because the method must track both which coins may be fake and whether each possible fake is lighter or heavier.

(b) **Verify the complexities without the Master Theorem.**

Tiling algorithm's divide-and-conquer approach.

Let $T(n)$ be the time to tile an $n \times n$ board. Each recursive call places one central tile, then solves four smaller boards of side length $n/2$. Assuming the missing-cell coordinates are available in each call, the nonrecursive work takes constant time. Thus,

$$T(n) = 4T(n/2) + c,$$

where c is a constant and $T(2) = d$ for a constant d.

In a calculation almost identical to Question 3, expand the recurrence:

$$\begin{aligned} T(n) &= 4\left(4T(n/4) + c\right) + c \\ &= 4^2 T(n/4) + 4c + c \\ &= 4^3 T(n/8) + 4^2 c + 4c + c. \end{aligned}$$

After j expansions,

$$T(n) = 4^j T(n/2^j) + c\sum_{i=0}^{j-1} 4^i.$$

Set $j = \log_2 n - 1$, so that $n/2^j = 2$ and $4^j = n^2/4$. Then

$$\begin{aligned} T(n) &= \frac{n^2}{4}T(2) + c\sum_{i=0}^{\log_2 n-2} 4^i \\ &= \frac{n^2}{4}d + c\frac{4^{\log_2 n-1} - 1}{3} \\ &= \frac{n^2}{4}d + c\frac{n^2/4 - 1}{3} \\ &= O(n^2). \end{aligned}$$

The algorithm must fill $\Theta(n^2)$ cells, so its running time is also $\Omega(n^2)$. Therefore,

$$\boxed{T(n) = \Theta(n^2).}$$

Fake-coin algorithm. Let $W(n)$ count balance weighings, with each weighing counted as one operation. For $n = 3^k$, when the fake is known to be lighter or heavier,

$$W(n) = W(n/3) + 1, \qquad W(1) = 0.$$

Expanding,

$$\begin{aligned} W(n) &= W(n/3^2) + 2 \\ &= W(n/3^3) + 3. \end{aligned}$$

After j expansions,

$$W(n) = W(n/3^j) + j.$$

Set $j = \log_3 n$. Then

$$W(n) = W(1) + \log_3 n = \log_3 n.$$

For arbitrary positive integer n, (which may not be divisible by 3) the worst-case count is

$$\boxed{W(n) = \Theta(\log n).}$$

This count is optimal: each weighing has three outcomes, so w weighings distinguish at most 3^w possibilities. Thus,

$$3^w \geq n \quad \Longrightarrow \quad w \geq \log_3 n.$$

When the fake may be either lighter or heavier, Egan's method tracks both possibilities and still uses $\Theta(\log n)$ weighings.

Therefore, for arbitrary n,

$$\boxed{W(n) = \Theta(\log n).}$$

(c) **Divide and Conquer versus Decrease and Conquer.**

In the tiling problem, the board is divided into four smaller boards, and the algorithm solves all four. This is **Divide and Conquer**.

In the fake-coin problem, each weighing leaves one smaller group that may contain the fake. The algorithm continues with that group and discards the others. This is **Decrease and Conquer**.

The key difference is that Divide and Conquer solves several smaller subproblems, while Decrease and Conquer solves only one.

5. n^2 trees of different heights grow in a spacious, arable $n \times n$ grid. At least one tree is shorter than all their neighbors. Given matrix H, where H[i, j] is the height of the tree in row i, column j, **design an efficient solution to find one tree overshadowed by its neighbors.** Use a consistent definition of 'neighbor' – e.g. trees 1 unit apart (horizontal and vertical but not diagonals) implies each corner tree (@grid positions [1,1], [1,n], [n,1], [n,n]) would have two neighbors; alternatively, if trees 1 unit apart include diagonals, each corner tree would have three neighbors. Differences also apply to border and interior cells.

(a) **Algorithm and pseudocode.**

Use horizontal and vertical neighbors only.

Find the shortest tree in the middle column and compare it with the saved candidate. Select the shorter of the two. For the first iteration, use the tree at $(1, 1)$ as the candidate.

Check all existing neighbors of the selected tree. If none is shorter, return it. Otherwise, keep the half containing a shorter neighbor and save that neighbor as the new candidate.

Repeat until a tree is shorter than all its neighbors. Keeping the candidate prevents the search from crossing an earlier dividing column.

```
FindShortTree(H, n, m):
    left = 1
    right = m
    best = (1, 1)

    while left <= right:
        middle = floor((left + right) / 2)

        for row = 1 to n:
```

```
        if H[row, middle] < H[best]:
            best = (row, middle)

    next = best
    for each existing neighbor of best:
        if H[neighbor] < H[next]:
            next = neighbor

    if next == best:
        return best

    if next.column < middle:
        right = middle - 1
    else:
        left = middle + 1

    best = next
```

Here, n is the number of rows and m is the number of columns. The neighbors are the existing cells directly above, below, left, and right. This pseudocode is helpful because we scan for neighbors with `H[neighbor]` but the actual code may be more convoluted.

How the algorithm works. At the start of each iteration, `best` (in the subsequent explanation we refer to `H[best]` as `best`, since `best` is an index pair) is shorter than every tree in a discarded column directly bordering the current region. Initially, there are no discarded columns.

After the column scan, `best` is no taller than its previous value or any tree in the middle column. Thus, if `next` is a shorter neighbor, it cannot be in a discarded bordering column or in the middle column. It must live in one of the two remaining halves.

The algorithm keeps that half and sets `best` to `next`. Every tree in the newly discarded middle column is taller than this new candidate, so the same condition holds for the next iteration.

Each iteration reduces the number of columns. When `next == best`, no neighbor is shorter. Since all heights are different, `best` is strictly shorter than all its neighbors, and we have found the locally shortest tree!

(b) **Analysis.**

(i) **Time complexity.**
Each iteration scans n trees in the middle column and checks at most four neighbors. Therefore, each iteration takes $\Theta(n)$ time. The number of remaining columns is reduced by approximately one-half:

$$m, \quad \frac{m}{2}, \quad \frac{m}{4}, \quad \dots$$

After j reductions, at most $m/2^j$ columns remain. Setting

$$\frac{m}{2^j} = 1$$

gives $j = \log_2 m$. Thus, there are at most $O(\log m)$ iterations, giving

$$\boxed{T(n, m) = O(n \log m)}$$

for $m \geq 2$.
This worst-case bound is tight: heights can decrease from left to right, forcing the search to keep the right half until the last column. Each iteration still scans all n rows. Therefore,

$$\boxed{T(n, m) = \Theta(n \log m).}$$

For the square grid, $m = n$, so

$$\boxed{T(n, n) = \Theta(n \log n).}$$

(ii) **Optimality of this approach.**
My research into this problem showed that this approach is not asymptotically optimal for a square grid. A method that searches both the middle row and middle column can reduce both dimensions by approximately one-half per step. Its total work is

$$O\left(n + \frac{n}{2} + \frac{n}{4} + \cdots\right) = O(n),$$

which is much more efficient than $\Theta(n \log n)$.

(iii) **Underlying paradigm.**
The method uses **Decrease and Conquer by a constant factor**.
At each iteration, it divides the current columns into two halves but continues with only one half. It discards the other half without solving it, and there are no separate results to combine. Therefore, under the distinction in Question 4, this is decrease and conquer.
The loop performs the same sequence of reductions that a recursive implementation would perform.